Digi Health

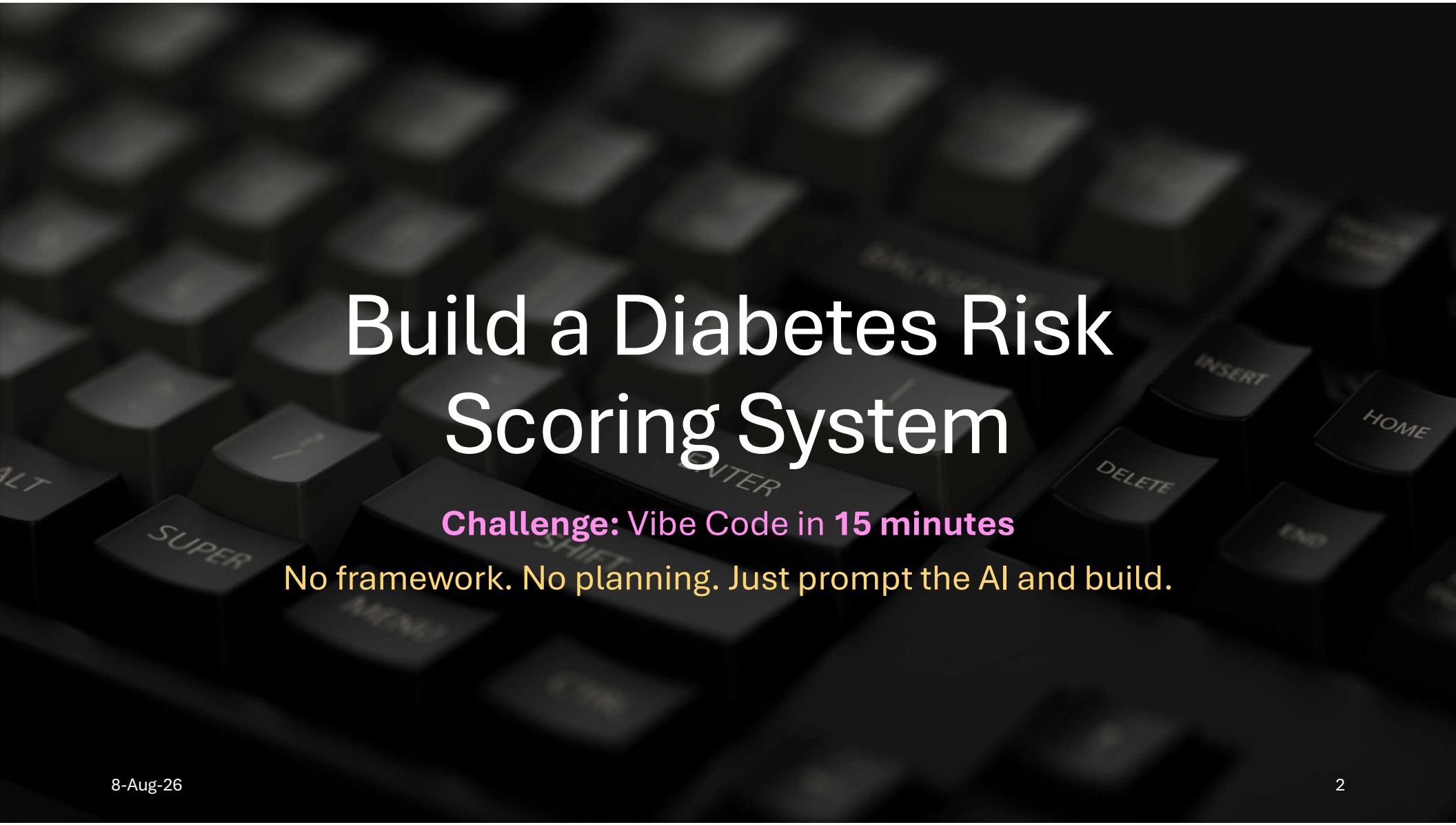
Vibe Coding 101

From Idea to Code

https://bit.ly/vibe-coding-101

Asst Prof Dr Naruemon Pratanwanich (Ploy)

คณะวิทยาศาสตร์
FACULTY OF SCIENCE
Chulalongkorn University



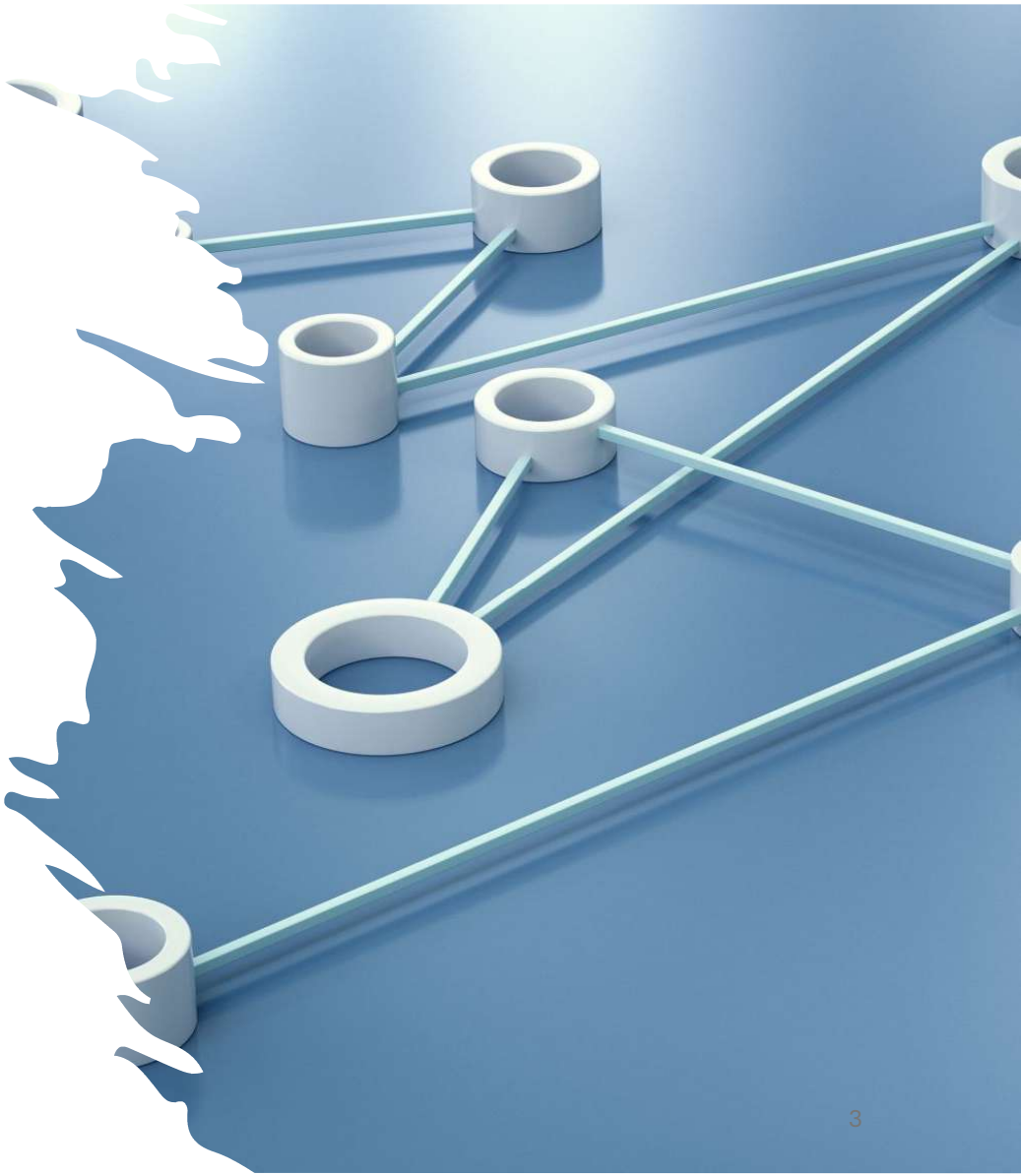
Build a Diabetes Risk Scoring System

Challenge: Vibe Code in **15 minutes**

No framework. No planning. Just prompt the AI and build.

Software Development Life Cycle (SDLC)

CLO1 Recognize the key stages of the Software Development Life Cycle (SDLC) and how AI can support each stage.



The Risks of “Vibe Coding” with Vague Prompts

When you replace precise engineering constraints with vague "vibes," AI guesses the implementation details.

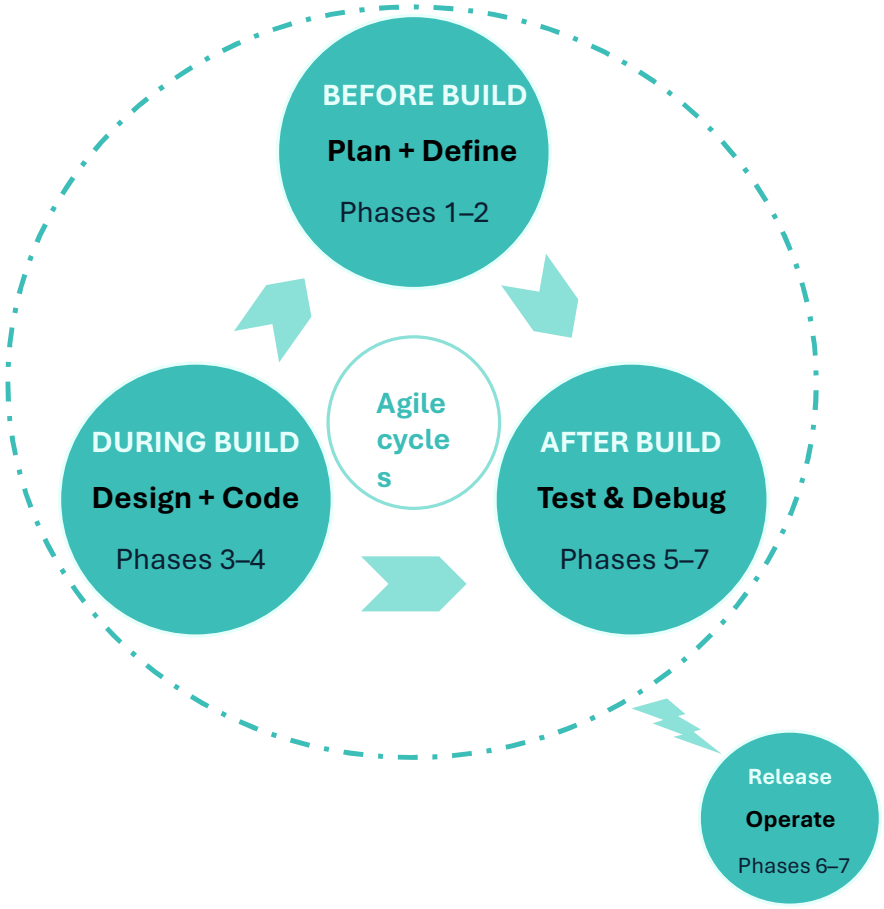
- **Architectural Spaghetti:** Non-modular code that is impossible to scale.
- **Context Blindness:** Redundant, chatty code rewrites rapidly exhaust the AI's memory limit.
- **Hidden Technical Debt:** Missing edge cases, absent error handling, and severe security gaps.
- **Loss of the Mental Map:** Hard to debug when it inevitably breaks.
- **The Hallucination Loop:** Vague error reporting creates a spiral of progressively broken code.

The 7 Phases of the Software Development Life Cycle (SDLC)

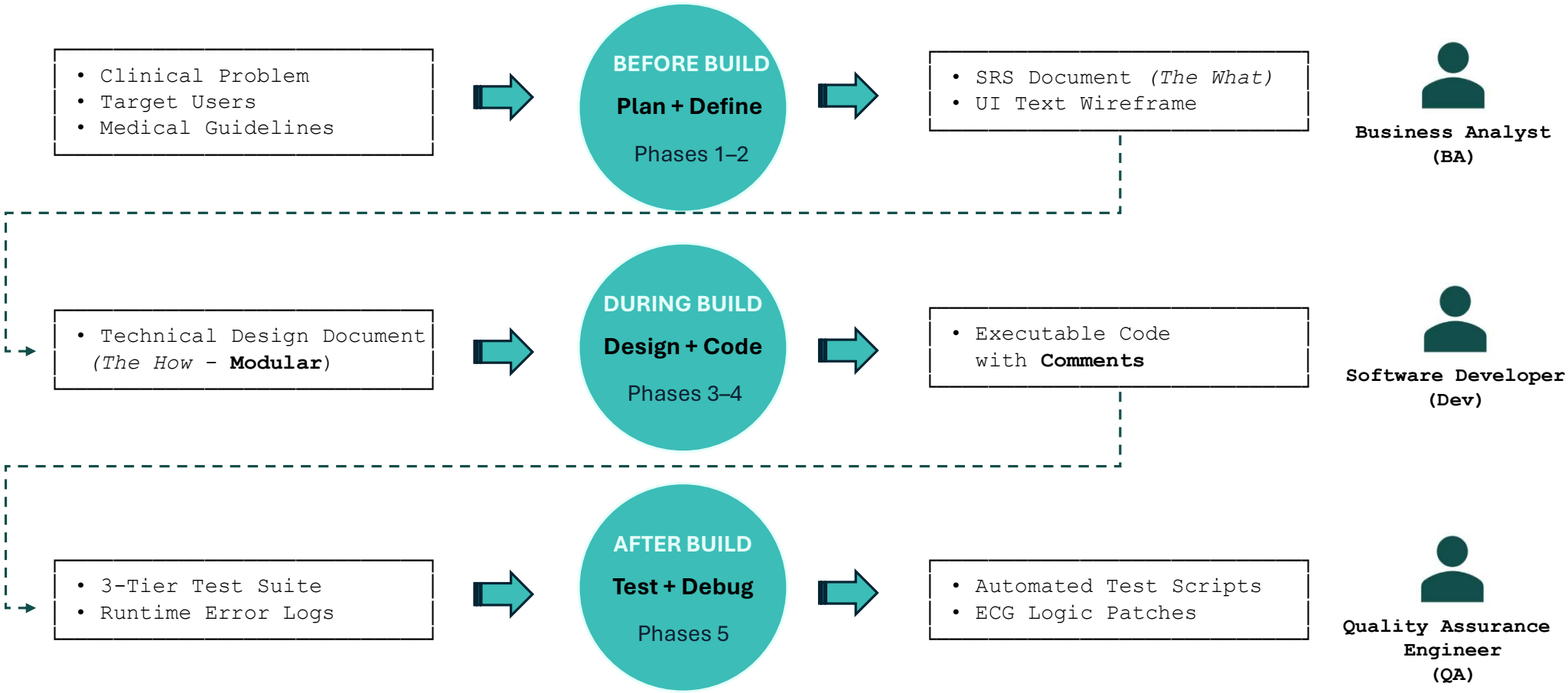
#	PHASE	GOAL	WHAT HAPPENS
1	**Planning & Requirement Analysis**	Figure out **what** we are building and **why**.	Meet stakeholders, calculate costs, define the audience, identify risks, and test feasibility.
2	**Defining Requirements**	Document everything the software must do.	Translate user needs into technical requirements and compile the **SRS** as the team's source of truth.
3	**Architectural Design**	Figure out **how** we are going to build it.	Choose the **tech** stack, database design, system architecture, modules, and APIs.
4	**Software Development**	Actually **build** the thing.	Developers write code from the design blueprint, usually through smaller tasks or sprints.
5	**Testing & Quality Assurance**	Find what is broken before users do.	QA **tests** bugs, security vulnerabilities, performance bottlenecks, and fit to Phase 2 requirements.
6	**Deployment**	**Release** the software to the world.	Stable code is pushed to production all at once, as a beta, or through a rolling release.
7	**Maintenance & Operations**	Keep the software alive and well.	Fix new bugs, optimize performance, and release updates or features from user feedback.

The 7 Phases of the Software Development Life Cycle (SDLC)

#	PHASE	GOAL
1	**Planning & Requirement Analysis**	Figure out **what** we are building and **why**.
2	**Defining Requirements**	Document everything the software must do.
3	**Architectural Design**	Figure out **how** we are going to build it.
4	**Software Development**	Actually **build** the thing.
5	**Testing & Quality Assurance**	Find what is broken before users do.
6	**Deployment**	**Release** the software to the world.
7	**Maintenance & Operations**	Keep the software alive and well.



Software Development Pipeline



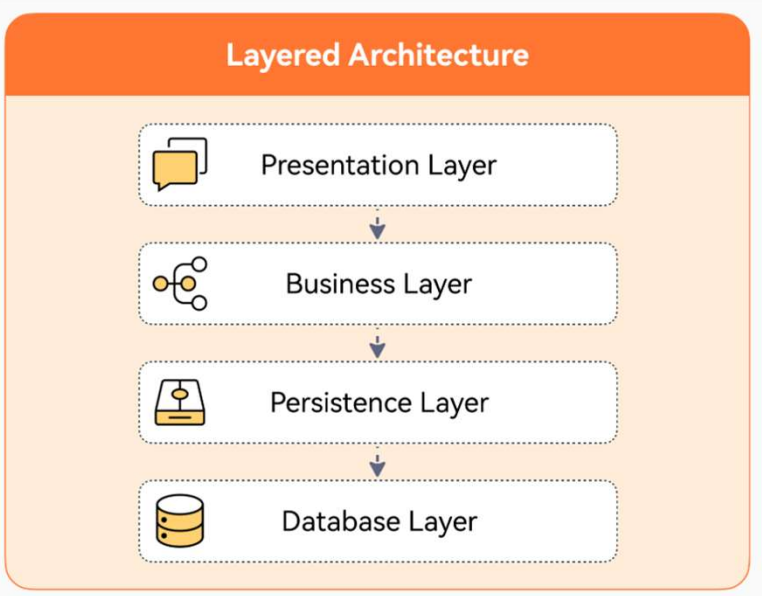
SRS: Software Requirement Specification

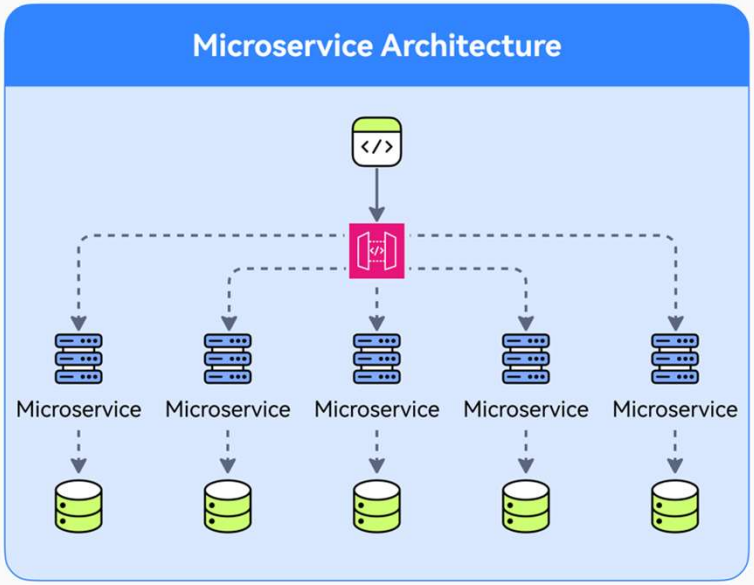
- **Business Goals & Scope ("Why")**
  - Problem statement, success metrics, and budget/deadlines.
  - Strict boundaries on what is **in-scope** vs. **out-of-scope**.
- **User Roles & Audience ("Who")**
  - Detailed personas (Primary users, Admins, Third-party actors).
  - Role-Based Access Control (RBAC) and permission matrices.
- **Functional Requirements ("What")**
  - Step-by-step user workflows and core feature capabilities.
  - Data inputs, processing rules, and expected system outputs.
  - Error handling, validation rules, and automated notification triggers.
- **Non-Functional Requirements ("How Well")**
  - **Performance:** Maximum page load times and API latency limits.
  - **Availability:** Target uptime percentages (e.g., 99.9% for medical apps).
  - **Scalability & Usability:** Storage growth plans and accessibility compliance.
- **Security & Compliance (The Guardrails)**
  - Regulatory requirements (HIPAA, GDPR, PCI-DSS).
  - Data encryption protocols (at-rest and in-transit).
  - Permanent audit logging for sensitive user actions.
- **System Integrations ("Connections")**
  - Third-party API dependencies and legacy system connections.
  - Target deployment environments, hardware dependencies, and browser support.

TDD: Technical Design Document

- Architectural Design

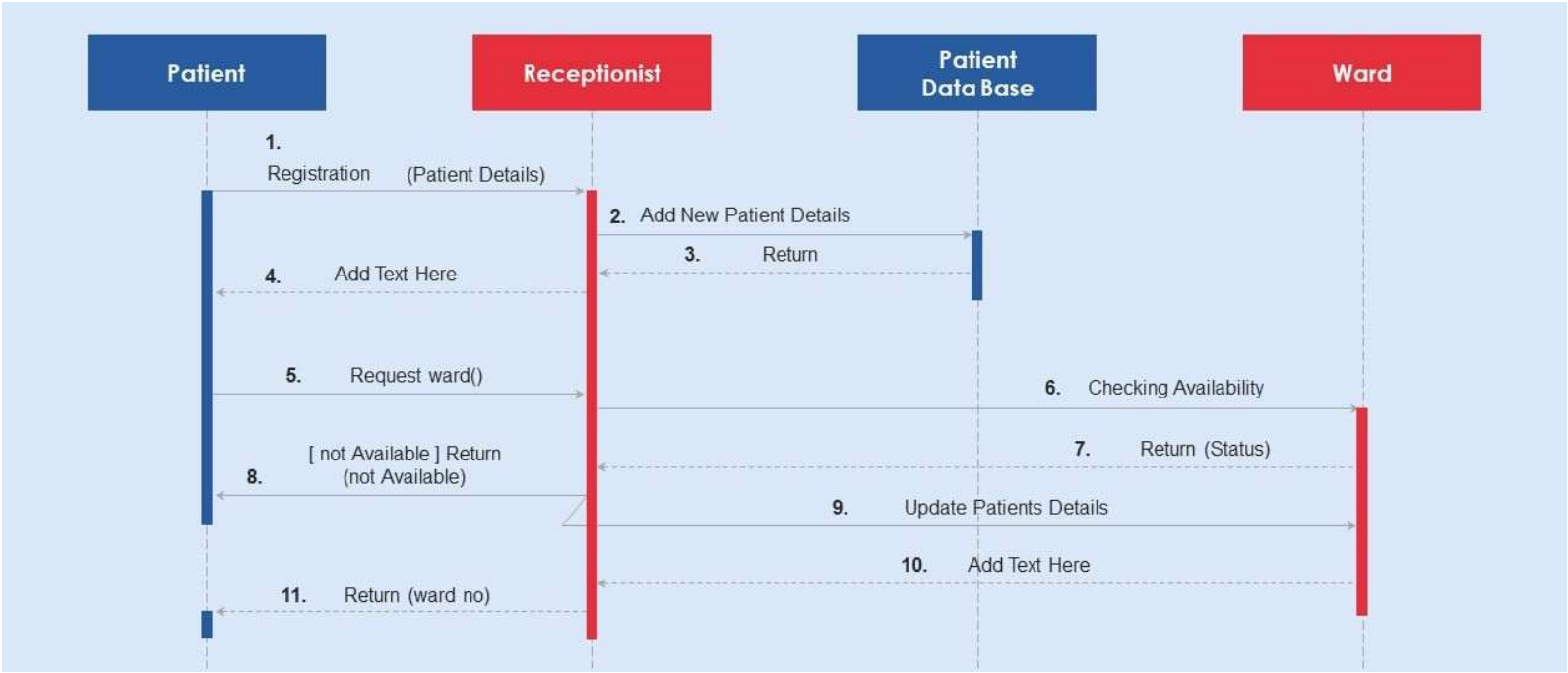
Modular





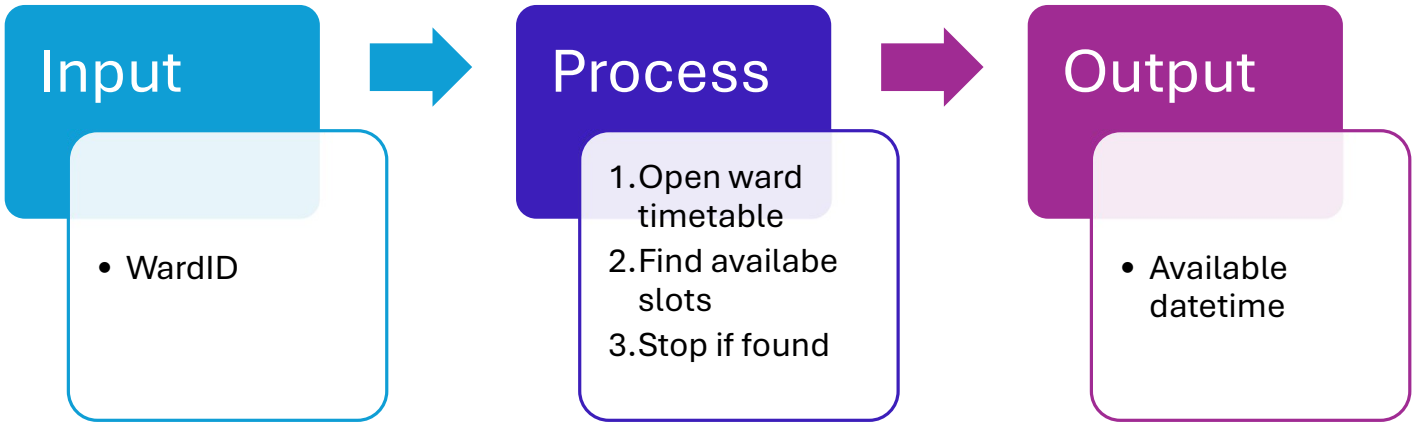
TDD: Technical Design Document

- Sequence Diagram



TDD: Technical Design Document

- Method Specification



3-Tier Test Suite

Unit Test

Verifies that an **individual function** or component behaves correctly in isolation

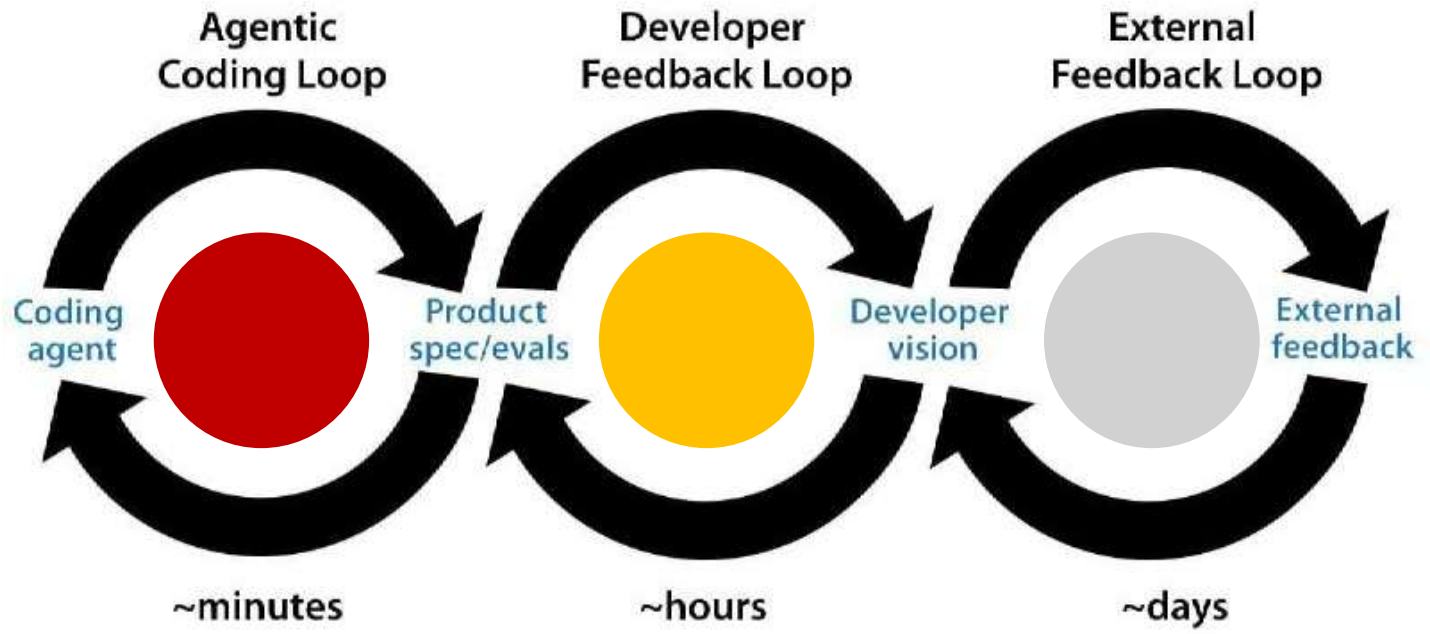
E2E Test

Verifies that the complete system works correctly by testing the **entire user workflow** from start to finish

Performance Test

Evaluates execution **time and memory** usage under large data loads or heavy workloads

AI-Assited Software Development Loops



Vibe Coding 101

Diabetes Risk Scoring App

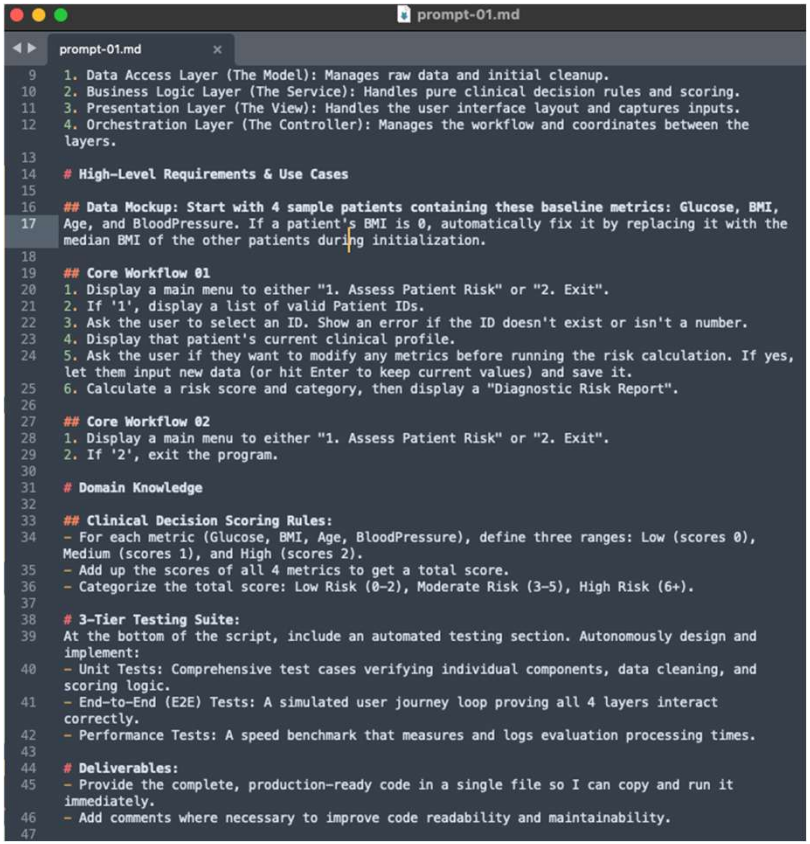
A clinical decision-support tool that processes metabolic data points (Glucose, BMI, Blood Pressure) to instantly classify patient diabetes risk profiles.

AI Coding Loop

CLO2 Use structured prompts to guide AI in designing and building console and GUI applications using layered architecture with a basic 3-tier testing suite.

SRS Based Prompt

- Role
- Context
- Architecture
- High-Level Requirements & Use Cases
  - Data Mockup
  - Core Workflow #1
  - Core Workflow #2
  - …
- Domain Knowledge / Rules
- Deliverables



SRS Based Prompt

 # Role

Act as an expert software architect and developer.

 # Context

I need to build a "Diabetes Risk Scoring System". I want you to design, structure, and fully implement a complete, working Python program based on the 4-Tier Architecture as follows.

Layered Architecture

 # Architecture

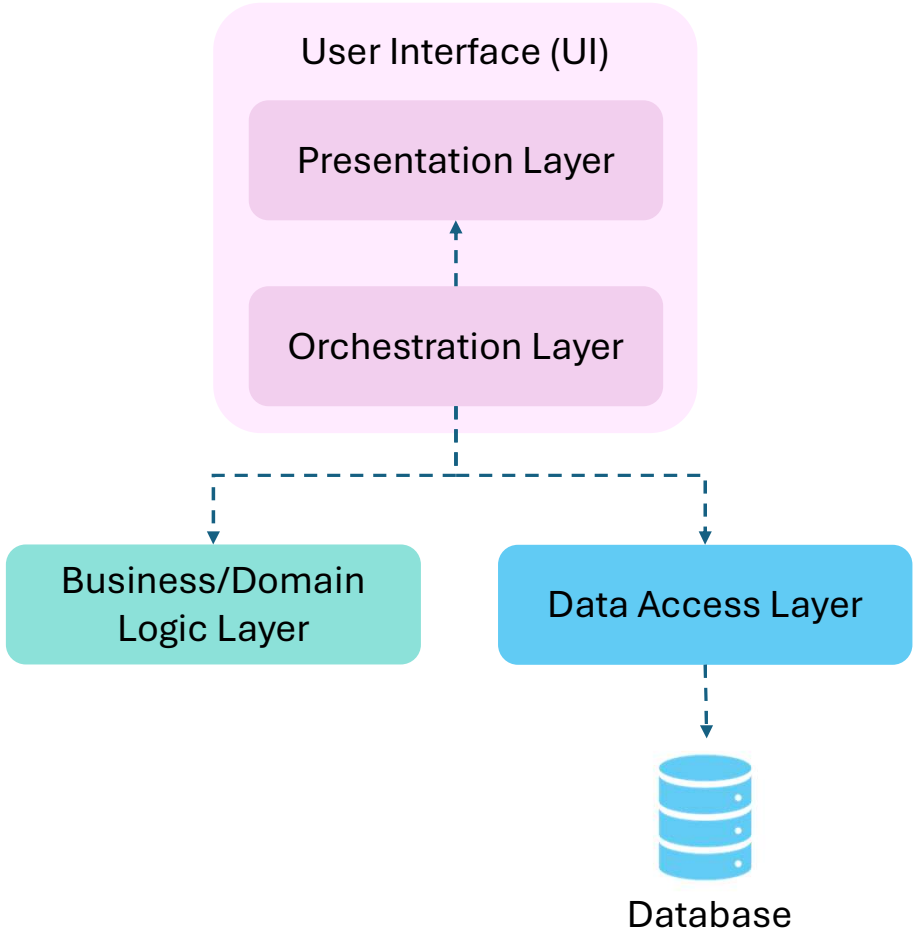
Separate the code cleanly into these 4 distinct layers:

1. *Data Access Layer:* Manages raw data and initial cleanup.

2. *Business Logic Layer:* Handles pure clinical decision rules and scoring.

3.1 *Presentation Layer:* Handles the user interface layout and captures inputs.

3.2 *Orchestration Layer:* Manages the workflow and coordinates between the layers.



SRS Based Prompt

 # High-Level Requirements & Use Cases

Data Mockup: Start with 4 sample patients containing these baseline metrics: Glucose, BMI, Age, and BloodPressure. If a patient's BMI is 0, automatically fix it by replacing it with the median BMI of the other patients during initialization.

Core Workflow 01:

1. Display a main menu to either "1. Assess Patient Risk" or "2. Exit".

2. If '1', display a list of valid Patient IDs.

3. Ask the user to select an ID. Show an error if the ID doesn't exist or isn't a number.

4. Display that patient's current clinical profile.

5. Ask the user if they want to modify any metrics before running the risk calculation. If yes, let them input new data (or hit Enter to keep current values) and save it.

6. Calculate a risk score and category, then display a "Diagnostic Risk Report".

Core Workflow 02:

1. Display a main menu to either "1. Assess Patient Risk" or "2. Exit".

2. If '2', exit the program.

SRS Based Prompt

 # Domain Knowledge

Clinical Decision Scoring Rules:

- For each metric (Glucose, BMI, Age, BloodPressure), define three ranges: Low (scores 0), Medium (scores 1), and High (scores 2).

- Add up the scores of all 4 metrics to get a total score.

- Categorize the total score: Low Risk (0-2), Moderate Risk (3-5), High Risk (6+).

Any further improvements ?

 # 3-Tier Testing Suite

At the bottom of the script, include an automated testing section. Autonomously design and implement:

- ***Unit Tests:*** Comprehensive test cases verifying individual components, data cleaning, and scoring logic.

- ***End-to-End (E2E) Tests:*** A simulated user journey loop proving all 4 layers interact correctly.

- ***Performance Tests:*** A speed benchmark that measures and logs evaluation processing times.

SRS Based Prompt

 # Deliverables

- Provide the complete, production-ready code in a single file so I can copy and run it immediately.

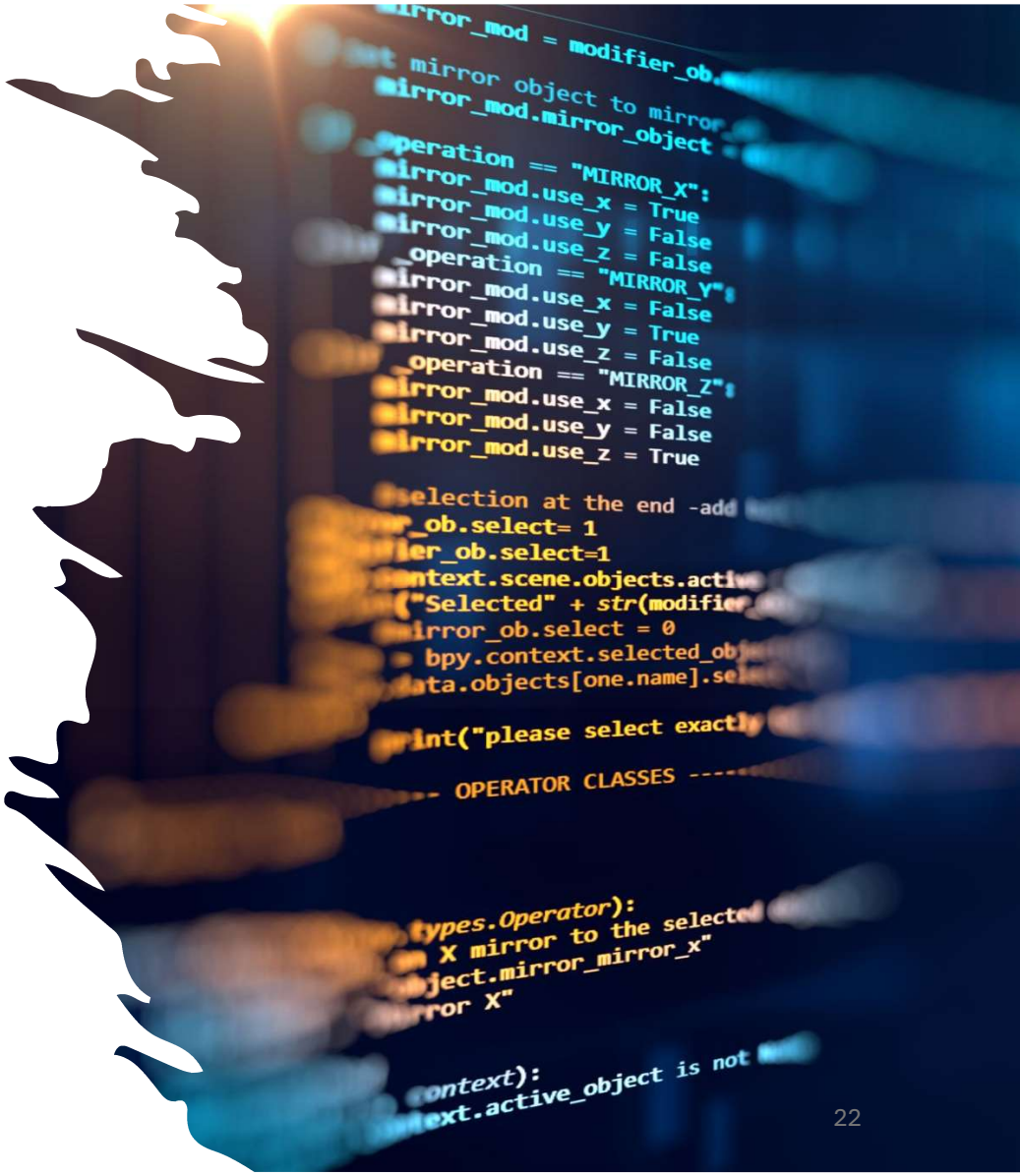
- Include a RUN_TEST_SUITE switch to toggle between running the 3-tier testing suite and launching the application.

- Add comments where necessary to improve code readability and maintainability.

Action required: Try prompt01-SRS.md > Run in Colab

Developer Feedback Loop

CLO3 Verify AI-generated code using structured Technical Design Documents (TDDs) generated via AI prompts.



Three Essential Programming Concepts

1. Algorithm: The step-by-step to complete a task

- **Data**
  - **Variables:** Labeled storage boxes used to hold data in memory.

```
x = {"BMI":25.1,"Age":39}
```

  - **Types:**
    - **Number:** int/float

```
1
```

    - **Text:** String

```
"1"
```

    - **List:** Collection of items in order

```
[1,2,3,4]
```

    - **Dictionary:** Collection of key-value pairs

```
{"BMI":25.1,"Age":39}
```

- **Selection:** Making decisions in code

```
if user is logged in
    show dashboard
else
    show login screen
```

- **Repetition (Loops):** Repeating an action multiple times until a specific condition is met.

```
sum(for x in range(1,5))
```

Three Essential Programming Concepts

2. Function/Method: A reusable block of code designed to perform a specific action

- **Parameter(s):** The **input** slots or placeholders a function requires to do its job
- **Return Value(s):** The final **output** or result that the function spits back out to the main program after it finishes running.
- **Function / Method Calling:** Triggering or executing that specific block of code to run when needed.

```
def f(x):
    y = x+1
    return y

def mean(x:list, w:list) -> float:
    for i in range(len(x)):
        weighted_sum += x[i]*w[i]
    return weighted_sum/sum(w)

mean([10,20,30],[1,2,1])
```

Three Essential Programming Concepts

3. Class: A custom blueprint or template used to define the properties and behaviors of a specific real-world concept.

- **Data (Attributes):** The information an object holds or remembers
- **Behaviour (Methods):** The actions an object can perform or the operations it can execute on its data
- **Object:** A real, live instance created from a **Class** blueprint
- **Object Instantiation:** The act of using a Class blueprint to actually create a live, concrete instance of that object in the program.

```
class Car:
    def __init__(mpg,fuel_level,make='Tesla'):
        self.make = make
        self.fuel_level = fuel_level
        self.mpg = mpg # miles per gallon
        self.pos = [0,0]
    def move(self,v,t):
        self.pos[0] += v[0]*t
        self.pos[1] += v[1]*t
    def calculate_range(self):
        range_miles = self.fuel_level*self.mpg

my_car = Car(20,30) # Object Instantiation
my_car.move(110.5,30)
```

Business Logic Layer
(The Service)

```python
class ClinicalRiskService:
    """Handles clinical decision rules, point scoring, and risk categorization."""

    THRESHOLDS = {
        "Glucose": (100.0, 125.0),
        "BMI": (25.0, 29.9),
        "Age": (35.0, 55.0),
        "BloodPressure": (120.0, 130.0)
    }

    def calculate_metric_score(self, metric_name: str, value: float) -> int:
        if metric_name not in self.THRESHOLDS:
            return 0
        low_max, med_max = self.THRESHOLDS[metric_name]
        if value <= low_max:
            return 0
        elif value <= med_max:
            return 1
        return 2
```

Usage

```python
service = ClinicalRiskService()
service.calculate_metric_score("Glucose", 95.0)
service.calculate_metric_score("BMI", 25.1)
```

Business Logic Layer
(The Service)

```python
class ClinicalRiskService:
    """Handles clinical decision rules, point scoring, and risk categorization."""

    THRESHOLDS = {
        "Glucose": (100.0, 125.0),
        "BMI": (25.0, 29.9),
        "Age": (35.0, 55.0),
        "BloodPressure": (120.0, 130.0)
    }

    def calculate_metric_score(self, metric_name: str, value: float) -> int:
        if metric_name not in self.THRESHOLDS:
            return 0
        low_max, med_max = self.THRESHOLDS[metric_name]
        if value <= low_max:
            return 0
        elif value <= med_max:
            return 1
        return 2

    def evaluate_patient_risk(self, metrics: Dict[str, float]) -> Tuple[int, str]:
        total_score = sum(self.calculate_metric_score(m, v) for m, v in metrics.items())
        if total_score <= 2:
            category = "Low Risk"
        elif total_score <= 5:
            category = "Moderate Risk"
        else:
            category = "High Risk"
        return total_score, category
```

Usage

```python
service = ClinicalRiskService()
service.evaluate_patient_risk({"Glucose": 90.0, "BMI": 22.0, "Age": 30.0, "BloodPressure": 110.0})
```

AI-Assisted Developer Feedback Loop

1. **The Blueprint (TDD Generation)**
   - AI analyzes code to build a **Technical Design Document (TDD)**.
   - Includes **Sequence Diagrams** (workflow) and **Method Specsification** (input-process-output).
2. **The Reality Check (Alignment)**
   - Developer reviews and updates the TDD to match the code.
   - Ensures the high-level design stays aligned with the actual **Source of Truth** (the codebase).
3. **Code Review**
   - AI inspects the code to generate a **Code Review Report**.
   - Flags **Code Smells** (poorly written code) and **Technical Debt** (messy fixes that cause future issues).
4. **The Auto-Repair (Refactoring)**
   - AI uses the core review report to clean and optimize the codebase.
   - Performs **Refactoring**—improving internal code structure without changing how the software behaves.

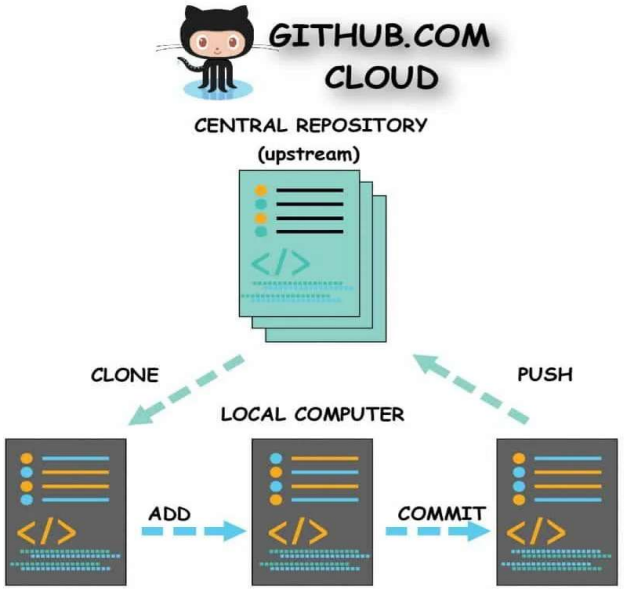
Action required:

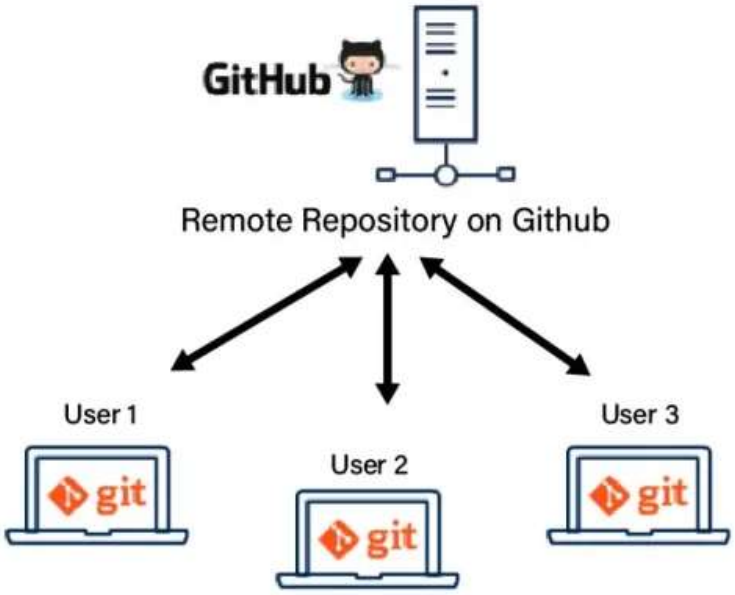
1. Try prompt02-TDD.md > Review Sequence Diagrams and Method Specification
2. Try prompt03-code-review.md > Review Issues

Basic GitHub

CLO4 Version and manage your application code and documentation using the basic features of GitHub.

Basic Github: Collaboration & Versioning



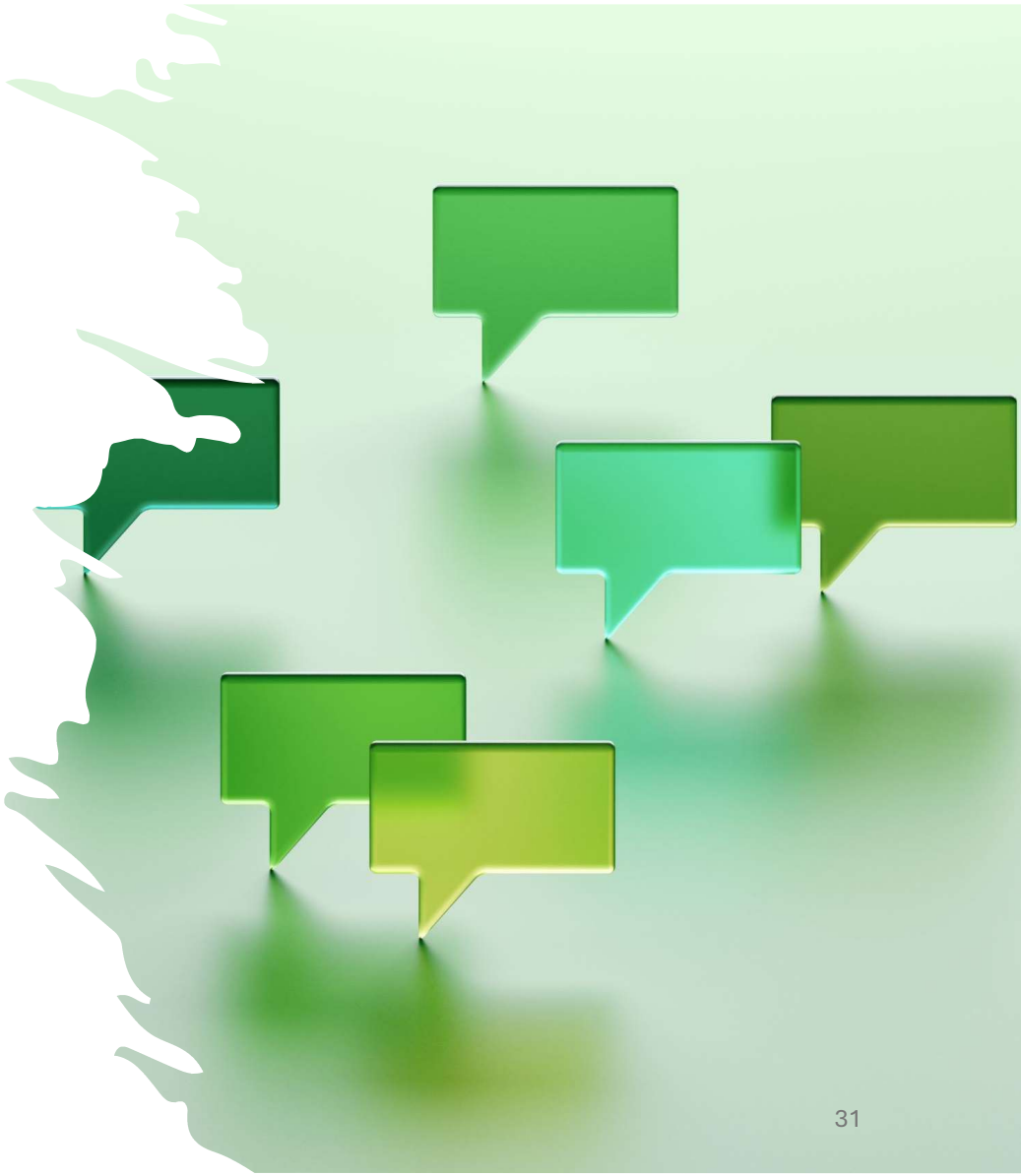


Action required:

1. Create your own GitHub Repository
2. Push your initial code and doc
3. Fix and test the code according to the issues in the code review report
4. Commit changes with proper comment

Iterative Application Improvements

CLO5 Iteratively improve applications through feedback-driven interactions with AI, enabling feature enhancements while preserving the existing codebase.



Making Changes I: Prompt

Targeted Feature Updates and Refinement

 # Role
Act as an expert software developer.

 # Context
I am updating the "Diabetes Risk Scoring" code we built. I only want to modify the specific function(s) responsible for the according to the user request below.

 # Instructions

1. Review the User Request below.
2. Locate the relavant function(s).
3. Update only that specific function(s) to match the requested visual style.
4. Output ONLY the updated function code. Do not reprint the rest of the application.

 # User Request
<PLACEHOLDER>

Exercise I

🟢 Easy: Single Method Change

User Request:

"I really like the application, but the diagnostic risk report printout at the end looks a bit plain. Can we make it look more like an official medical receipt? Add some extra decorative borders (like rows of hyphens or asterisks) and clearly print a 'CONFIDENTIAL' warning at the bottom so it stands out on the screen."

Action required:

1. Try prompt04-feedback-ex1.md
2. Test in Colab
3. Commit changes in GitHub

Exercise II

🟡 Medium: Multiple Methods, Single Class

User Request:

"Right now, the system just spits out a final score and a risk category. As a clinician, a single label isn't enough for me to explain things to my patients. I need the app to also give me a quick text summary explaining why they got that score—specifically, a breakdown showing which exact health metrics (like their Glucose or BMI) were actually outside the healthy boundaries."

Exercise III

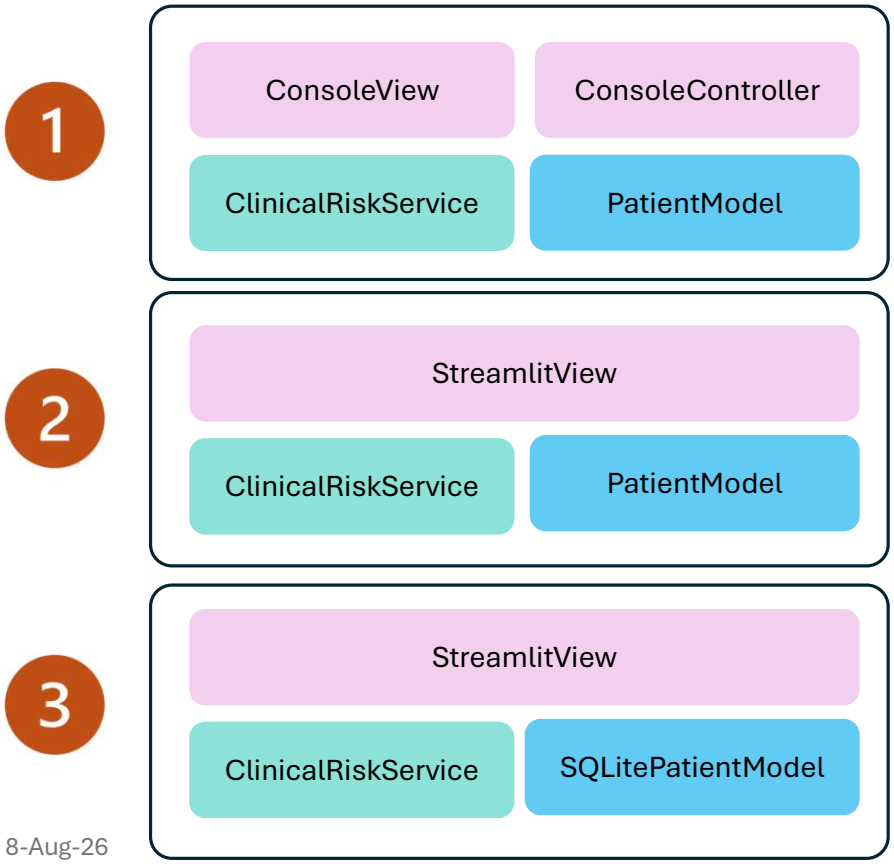
🔴 Hard: Cross-Tier Modification

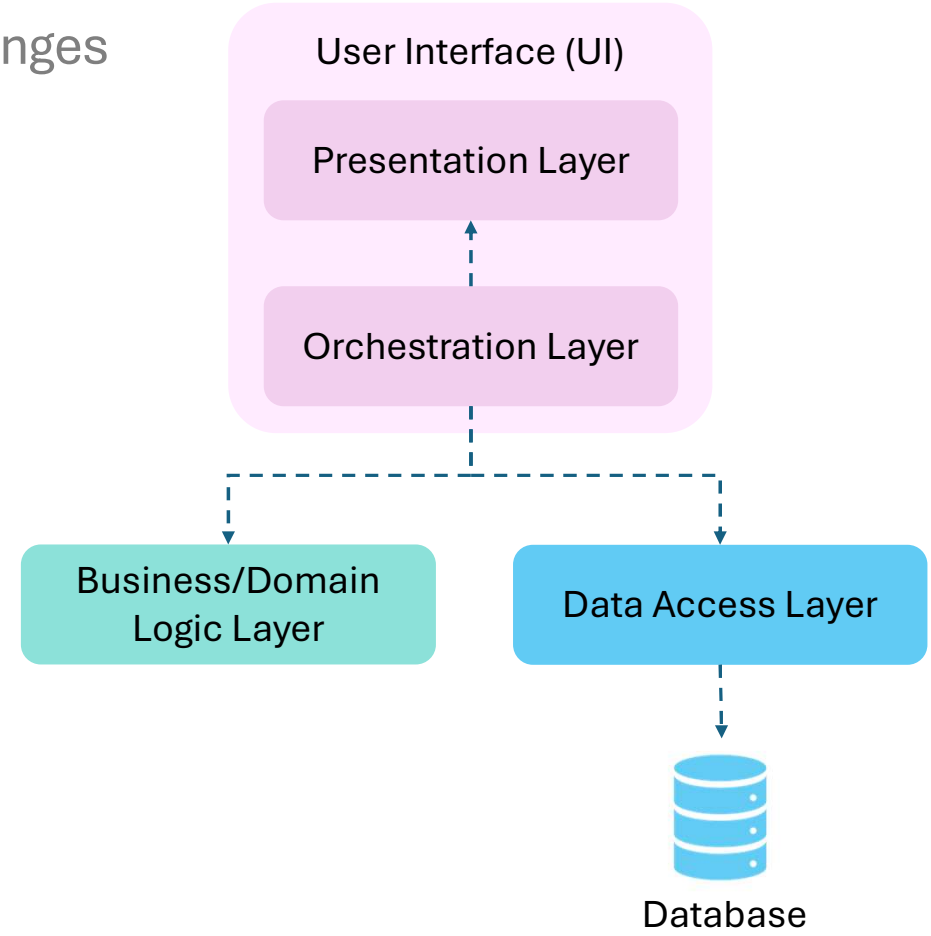
User Request:

"When looking at a patient's risk profile, seeing just a final category isn't completely informative. We need to see how their metrics compare to the average of our entire clinic clinic database. Can you modify the system to calculate the clinical average for each metric across all registered patients, and then display whether this specific patient is 'ABOVE', 'BELOW', or 'EQUAL' to that clinic average next to their numbers?"

Making Changes II: Prompt

Evolving the Architecture with Layered Changes





Exercise IV

🟡 Presentation Layer Modification

User Request:

"The current system works well, but interacting through the console is not very user-friendly. Can you convert the existing console-based interface into a graphical user interface (GUI) application? The new interface should provide visual components such as input fields, buttons, and display panels while preserving all existing functionalities and keeping the underlying business logic unchanged."

Action required: Prompt and run in Streamlit Playground

Exercise V

🔴 Data Access Layer Modification

User Request:

"The current system stores all patient information in memory, which means the data is lost whenever the application is closed. We need a more reliable way to manage patient records. Can you modify the system to store and retrieve patient data using a database instead of in-memory storage? The system should preserve all existing functionalities while replacing the current data storage mechanism with persistent database operations."

Action required: Prompt and run in Streamlit Cloud

Wrap-Up

Visual Studio Code
+ CoPilot / Codex